

Practical Deep Learning for Stock Trading

1. Overview

Return maximization is based on estimates of stocks' potential return and risk

A traditional approach for this is done in two steps

expected returns and covariance matrix are calculated
"on avg how much will each stock earn" "How stocks move together"
pos → up & down same time

Best portfolio allocation is found through

- 1) maximizing return for a fixed risk
- 2) minimizing the risk for range of returns

The best trading strategy is then extended by the best portfolio allocation (complicated)

Another approach to the stock trading prob is to model as a **Markov Decision Process (MDP)**
and to use Dynamic Programming to solve for optimum strategy (scalability limited)

Deep Deterministic Policy Gradient (DDPG)

Algo consists of 3 components:

- i) actor-critic framework that models large states and action spaces
- ii) target networks that stabilize training process
- iii) Experience replay that removes the correlations between samples and increases the usage of data

Note: higher returns on avg than broad min-var portfolio & Dow Jones Industrial avg (DJIA)

2. Problem Statement

We model a stock trading process as a MDP then formulate our trading goal as a **maximization problem**

Problem formulation "stochastic & random"

Consider the stochastic and interactive nature of trading markets, model the stock trading process as a MDP:

1. State $s = [p, h, b]$

a set that includes:

- p : the set of prices of stocks $p \in \mathbb{R}_+^D$
- h : The amount of holdings of stocks $h \in \mathbb{Q}_+^D$
- b : the remaining balance $b \in \mathbb{R}_+$
- D : number of stocks we consider in the market

2. Action a : Set of actions on all D stocks

available actions on each stock

- selling → increase $\rightarrow h$
- buying → decrease $\rightarrow h$
- holding → no change

3. Reward $r(s, a, s')$ change of the portfolio value when action a is taken at state s and arriving at new state s'

The portfolio is the sum of the equities in all held stocks $p^T h$ and balance b

4. Policy $\pi(s)$: Trading strategy of stocks at state s . (Essentially probability distribution of a at s)

"game plan"
decides what actions to take

example:

$\pi(s) = \{ \text{Buy: } 0.6, \text{ Hold: } 0.3, \text{ Sell: } 0.1 \}$
probability distribution (adds to one)

5. Action value function $Q_{\pi}(s, a)$: Expected reward achieved by action a at state s following policy π

- Long term reward if you take action a at state s and follow the policy
- It is about the future (EX: "If I buy AAPL now and keep trading with my strategy, what's the profit?")

equity = price $\times$ holdings
 $= p \cdot h$
as a vector (use dot product)
which can be rep as
 $= p^T h$

Portfolio can be desc as
 $= \text{equity} + \text{balance}$
 $\uparrow \quad \uparrow$
stock you hold left over cash
 $= p^T h + b$

The dynamics of the stock market:

Let t denote time and the available actions on stock d are

- selling: $K \in [1, h[d]]$, where $d=1, \dots, D$ shares can be sold from holdings
 $K \in \mathbb{Z}$ and $h_{t+1} = h_t - K$
- holding: $K=0$ and no change in h_t
- Buying: K shares bought
 $h_{t+1} = h_t + K$ ($a_t[d] = -K$ is neg. int)

Very obvious if you think about it

Note: stocks bought should not result in neg balance on portfolio

WLOG assume that selling orders are made on the first d_1 stocks and buying are made on the last d_2 ones and that

a_t satisfies

$$p_t[1:d_1]^T a_t[1:d_1] + b_t + p_t[D-d_2:D]^T a_t[D-d_2:D] \geq 0$$

Price of first d_1 stocks selling action for the first d_1 stocks balance at time t price of the last d_2 stocks buying action of the last d_2 stocks

Combine to make equality $\Rightarrow p_t^T a_t$

this then becomes

$$b_{t+1} = p_t^T a_t + b_t$$

(New balance updated from the buying and selling)

Initial conditions and how it progresses

Before being exposed to the "environment", p_0 is set to the stock prices at $t=0$ and b_0 is the initial fund before trading

The h and $Q_{\pi}(s, a)$ are initialized as 0 and $\pi(s)$ is uniformly distributed among all actions for any state

$Q_{\pi}(s_t, a_t)$ is learned through interacting with the environment



Lemma: The Bellman Equations

Used in Reinforcement learning to calculate the value of a state. The value of a state is equal to the reward received now plus the expected value of the next state.

It helps an agent make decisions by considering immediate and future rewards.

Based on the "principle of optimality" meaning the best value depends on immediate reward and the value of next state

State Value function is $V(s)$ under a given policy represents the expected cumulative reward starting from state s and following that policy

$$V^{\pi}(s) = E [R(s, a) + \gamma V^{\pi}(s')]]$$

$V^{\pi}(s)$: Value of state s under policy

$P(s'|s, a)$: Transition probability from state $s \rightarrow s'$

$R(s, a)$: Reward obtained after taking action a in state s when taking a

γ : Discount factor (controls importance of rewards)

$\pi(a|s)$: Prob of taking action a in state s under policy

Derivation:

We define the return (total future reward) starting from t

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

r_{t+1} : reward after first action, $r_{t+2}, \dots$: reward in future, γ : discount factor future rewards less than the present — think of inflation

Breaking into parts

$$G_t = r_t + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) = r_{t+1} + \gamma G_{t+1}$$

Recursive structure where we can use Dynamic Programming

Since $V_t^{\pi}(s) = E[G_t | s_t, a_t]$ ← Expected returns at state s_t given action a_t

Sub ① $V_t^{\pi}(s) = E[r_t + \gamma G_{t+1} | s_t, a_t]$

Sub ② $V_t^{\pi}(s) = E[r_t + \gamma V_{t+1}^{\pi}(s') | s_t, a_t]$ QED

Bellman Equation, the expected reward of taking action a_t is calculated by taking expectation of the rewards $r(s_t, a_t, s_{t+1})$ plus the expected reward in state s_{t+1}

$$Q_{\pi}(s_t, a_t) = E_{s_{t+1}} [r(s_t, a_t, s_{t+1}) + \gamma E_{\pi} [Q_{\pi}(s_{t+1}, a_{t+1})]]$$

Expected rewards immediate rewards Future rewards

Trading as a Return Maximization

The goal is to design a trading strategy that maximizes investment return at time t_f

in the future $P_t \text{Th} + b_t$, which is equiv to $\sum_{t=1}^{t_f-1} r(s_t, a_t, s_{t+1})$

Due to Markov property of the model, the problem can be solved as optimizing the policy that maximizes $Q_\pi(s_t, a_t)$

Deep learning Approach

Employing DDPG algo to maximize investment return.

DDPG is an improved ver of Deterministic Policy gradient (DPG) algo (which combines Q-learning and policy grad)

DDPG uses neural networks as function approximator

DDPG is specified for MDP model of the stock trading market

Q-learning is a method to learn the environment

Instead of using expectation of $Q(s_{t+1}, a_{t+1})$ to update $Q(s_t, a_t)$, it uses greedy action a_{t+1} that maximizes $Q(s_{t+1}, a_{t+1})$ for s_{t+1}

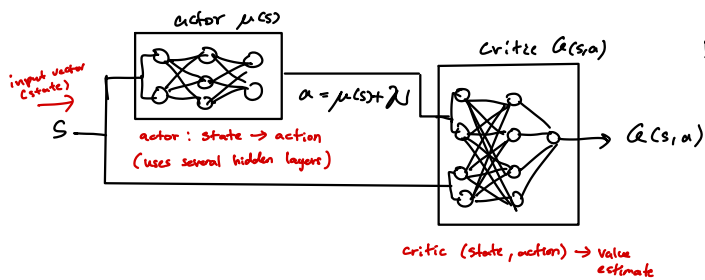
$$Q_\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}} (r(s_t, a_t, s_{t+1}) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}))$$

max term used optimal case for $Q_\pi(s_{t+1}, a_{t+1})$ "best possible future outcome"

With deep Q-network (DQN), which adopts neural networks to perform function approx, the states are encoded in the Val func

DQN is impractical since large size of action space (action space grows exp "curse of dimensionality")

DDPG is used to deterministically map states to actions to address this issue



Learning Network architecture

DDPG maintains an actor network and a critic network

- actor $\mu(s|\theta^\mu)$ maps states to actions where θ^μ is the set of actor network params
input: state s (portfolio, stock prices, etc) Purpose: To propose a strategy (what should I do?)
output: an action a (buy, sell, hold)
During training: noise is added to encourage "exploration"
- critic network $Q(s, a|\theta^Q)$ outputs the value of action under that state
input: state s and action a
output: predicted Q value (expected longterm reward)
purpose: To judge the strategy "was it good?"
- θ^Q is the set of critic network params
- Noise is added to the output of actor which is sampled from a random process $\mathcal{N}$
- DDPG uses an experience replay buffer $\mathcal{R}$ to store transitions and update the model

The main idea is the NETWORK learns a mapping

Algorithm: core idea $\rightarrow$ slowly update weights overtime using target (stochastic) weights and applying

1. initialize actor and critic network with random weights θ^Q and θ^μ
2. initialize target networks Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$ & $\theta^{\mu'} \leftarrow \theta^\mu$
3. Initialize replay buffer $\mathcal{R}$ - FIFO memory to store past transitions (s, a, r, s') , breaks correlation
4. for episode in range $(1, M)$:
5. initialize: random process $\mathcal{N}$ - for action exploration

Restore initial observation state s_1

for t in range $(1, T)$:

select $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to current policy & exploration noise

Execute action a_t and observe reward r_t and state s_{t+1}

store transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{R}$ - obvious

Sample a minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from $\mathcal{R}$

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))$

update critic by minimizing the loss $\mathcal{L} = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$ - update by minimizing Mean Square error

update the actor policy by using the sampled policy gradient

$$\nabla_{\theta^\mu} J = \frac{1}{N} \sum_i \left[\frac{\partial}{\partial a} Q(s_i, a|\theta^Q) \Big|_{s=s_i, a=\mu(s_i|\theta^\mu)} \frac{\partial}{\partial \mu} \mu(s_i|\theta^\mu) \Big|_{s_i} \right]$$

update the target networks

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1-\tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1-\tau) \theta^{\mu'}$$

- Polyak averaging, small $\tau \in (0, 1)$, slowly moving targets helps the updates

- self explanatory, creating trainable weights

- target network: slowly updated copy. The reason is so when training

both sides of the equation don't change at the same time, they provide stable reference values for updates. understanding:

Given $\mathcal{L}(\theta^Q) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2 \rightarrow$ use Q' and μ' for it to be more stable

You try to update Q and μ BUT both LHS and RHS change at the same time like a "moving target", to solve it we use "target" for RHS analogy: "chasing your shadow"

- action occurs with exploration

- reward is change in $P \text{Th} + b$ from $t \rightarrow t+1$

- randomly sample from the buffer since stochastic gradient descent becomes unstable/biased when data are highly correlated

- update actor: uses the "deterministic policy gradient theorem" the surrogate objective is the critic

intuition: change θ^μ so the actor outputs actions that the critic scores higher. (It is chain rule: increase Q by a)

This is what we plug into our SGD to update our actor's weight

TLDR: We want to maximize $\mathbb{E}(Q(s, \mu(s)))$, so we perform a SGD to "climb" the hill relative to the actor's parameters

actor learns by following a gradient of expected returns (DPG)
There is no "correct" buy or sell, we need to constantly improve

Critic learns by fitting to bottom target

Deterministic Policy gradient

1. goal: we want to maximize expected returns, in this case we use a surrogate goal to maximize our *estimated* expected returns

$$J(\theta^{\mu}) = \mathbb{E}_{s \sim D} [Q(s, \mu(s, \theta^{\mu}))] \rightarrow J \text{ can be thought of as}$$

- $s \sim D$ sampled states from $\mathcal{R}$ *on average how good are my actions that the actor output*
- $Q(s, a)$ critic's estimate of long-term value *according to my critic*
- $\mu(s|\theta^{\mu})$ actor's deterministic policy (a)

2. Differentiate both sides w.r. θ^{μ}

$$\begin{aligned} \nabla_{\theta^{\mu}} J &= \nabla_{\theta^{\mu}} \mathbb{E}_{s \sim D} [Q(s, \mu(s, \theta^{\mu}))] && \text{Bring inside} \\ &= \mathbb{E}_{s \sim D} \left[\nabla_{\theta^{\mu}} (Q(s, \mu(s, \theta^{\mu}))) \right] && \text{from the chain rule} \\ &= \mathbb{E}_{s \sim D} \left[\nabla_{\theta^{\mu}} Q(s, \mu(s, \theta^{\mu})) \cdot \nabla_{\theta^{\mu}} \mu(s, \theta^{\mu}) \right] \end{aligned}$$

The critic tells us how much Q increases or decreases if action changes a bit ($\nabla_a Q$)

the actor tells us how its action shifts if we tweak its parameters $\nabla_{\theta^{\mu}} \mu$

Multiplying them: how to adjust actors' weights to increase Q